Basketball Injury Recurrence Prediction Project

Lana Robison & other group members

```
library(tidymodels)
library(tidyverse)

injury_dat = read_csv("basketball_injury_biomechanical_dataset.csv")
head(injury_dat)
```

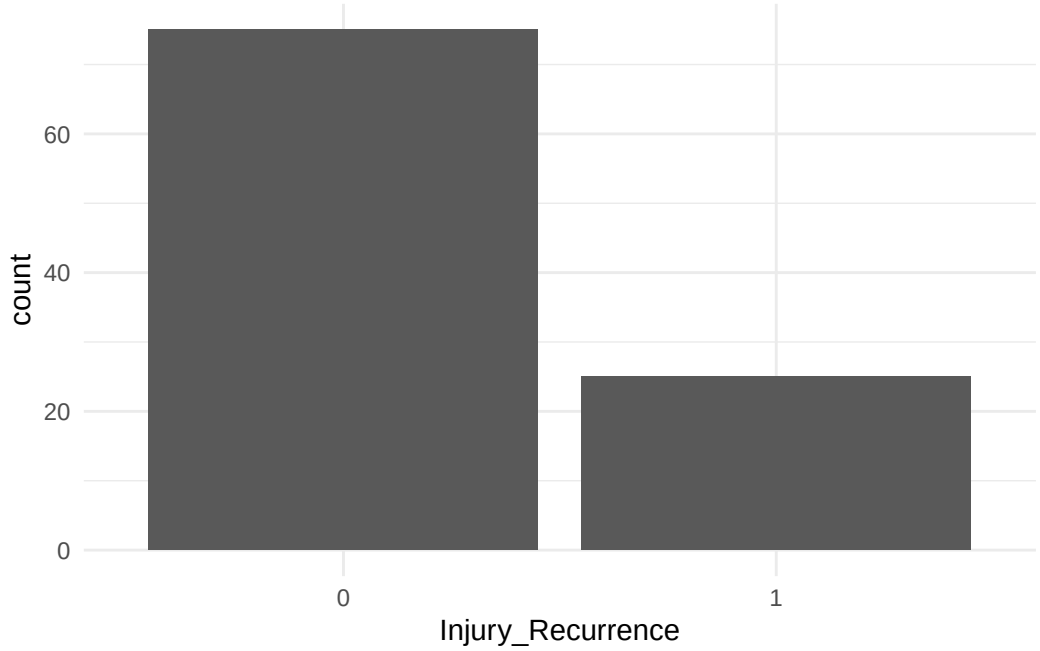
```
# A tibble: 6 x 17
  Player_ID   Age Height_cm Weight_kg Position Injury_Type       Injury_Severity
      <dbl> <dbl>     <dbl>     <dbl> <chr>    <chr>             <chr>
1         1    24       195       108 Center   ACL Tear          Severe
2         2    32       183        87 Forward  Ankle Sprain      Mild
3         3    28       208       109 Center   Knee Injury       Moderate
4         4    25       196        70 Forward  Shoulder Disloca~ Moderate
5         5    24       178        80 Guard    Shoulder Disloca~ Moderate
6         6    28       184        97 Guard    Ankle Sprain      Severe
# i 10 more variables: Rehabilitation_Program <chr>,
#   Rehabilitation_Time_weeks <dbl>, Injury_Recurrence <dbl>,
#   Date_of_Injury <date>, Rehabilitation_Efficiency_Score <dbl>,
#   knee_angle_deg <dbl>, jump_height_cm <dbl>, ankle_flexion_deg <dbl>,
#   speed_m_s <dbl>, reaction_time_ms <dbl>
```

Cleaning

```
# Convert categorical predictors to numeric
dat = injury_dat |> mutate(
  Position = as.numeric(factor(Position)),
  Injury_Type = as.numeric(factor(Injury_Type)),
  Injury_Severity = as.numeric(factor(Injury_Severity)),
  Rehabilitation_Program = as.numeric(factor(Rehabilitation_Program)),
```

```
  Injury_Recurrence = factor(Injury_Recurrence)
) |>
  select(
    -c(Player_ID, Date_of_Injury)
  )

# Visualization of Injury Recurrences
dat |> ggplot(aes(x = Injury_Recurrence)) +
  geom_bar() +
  theme_minimal() +
  scale_x_discrete()
```




```
set.seed(23)
dat_split = initial_split(dat, prop = 0.8)
dat_train = training(dat_split)
dat_test = testing(dat_split)
```

Random Forests Classification

```
# Define random forest model
ref_spec = rand_forest(mtry = tune(), min_n = tune(),
                       trees = 100) |>
  set_mode("classification") |>
  set_engine("ranger")

tunning_grid = grid_regular(
  mtry(range = c(5,14)),
  min_n(range = c(2, 40)),
  levels = 5
)

dat_folds = vfold_cv(dat_train, v = 3, strata = Injury_Recurrence)

# Tune the model using CV
rf_results = tune_grid(
  ref_spec,
  Injury_Recurrence~.,
  resamples = dat_folds,
  grid = tunning_grid,
  metrics = metric_set(accuracy)
)

# Select the best hyperparameters based on accuracy
best_parms = select_best(rf_results, metric = "accuracy")

# Fit final RF model
ref_spec = rand_forest(mtry = best_parms$mtry, min_n = best_parms$min_n,
                       trees = 100) |>
  set_mode("classification") |>
  set_engine("ranger")

ref_fit = ref_spec |>
  fit(Injury_Recurrence~., data = dat_train)

# Evaluate model on train data
probs_train = ref_fit |> predict(dat_train, type = "class") |>
  bind_cols(dat_train)

print(accuracy(probs_train, truth = Injury_Recurrence,
               estimate = .pred_class))
```

```
# A tibble: 1 x 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy binary             1
```

```
# Evaluate model on test data
probs_test = ref_fit |> predict(dat_test, type = "class") |>
  bind_cols(dat_test)

print(accuracy(probs_test, truth = Injury_Recurrence,
               estimate = .pred_class))
```

```
# A tibble: 1 x 3
  .metric  .estimator .estimate
  <chr>    <chr>          <dbl>
1 accuracy binary           0.6
```

We chose a random forests model because it is able to handle both numerical and categorical predictors, and because it generally performs well on datasets with non-linear relationships and high dimensionality. We also felt that for this project we wanted to have a high predictive accuracy, which a random forests model is more prone to provide compared to a logistic regression model.

Model Accuracy Results

Dataset	Accuracy
Train	0.81
Test	0.65

Our model is overfitting the data, it performs exceptionally well on the training data, but poorly on the test data. This means it has memorized training patterns that do not generalize well.

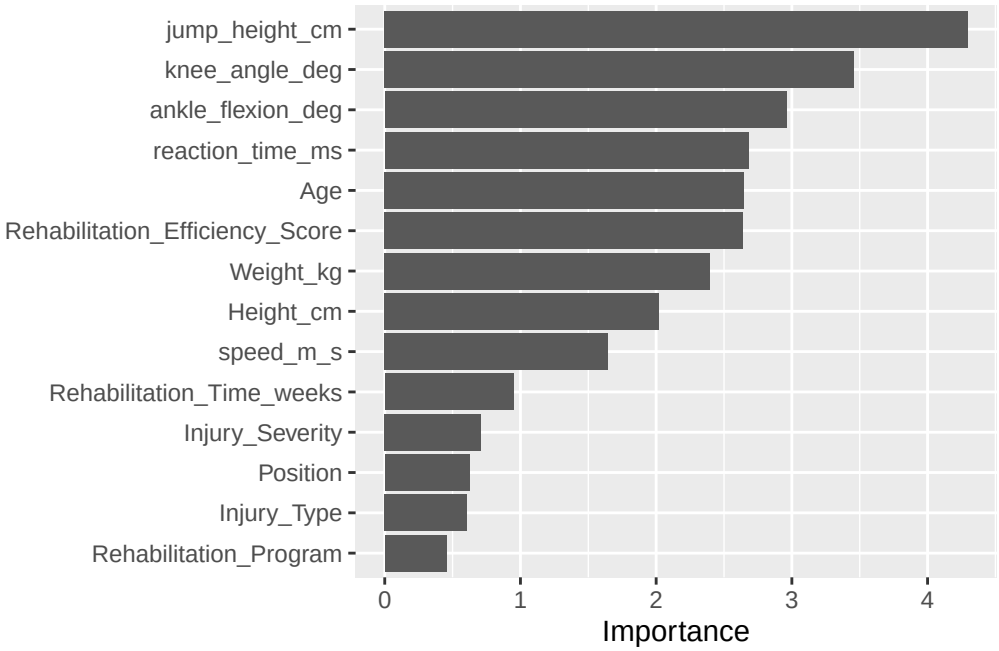
```
library(vip)

# refit with importance enabled for variable ranking
ref_spec = rand_forest(mtry = best_parms$mtry,
                       min_n = best_parms$min_n,
                       trees = 100) |>
```

```
  set_mode("classification") |>
  set_engine("ranger", importance = "impurity")

ref_fit = ref_spec |>
  fit(Injury_Recurrence~., data = dat_train)

# Visualize variable importance
vip(ref_fit$fit, num_features = 20)
```



According to the above variable importance plot, the most important predictors for injury recurrence were "knee_angle_deg" (player's knee angle in degrees during basketball movements), "jump_height_cm" (height the player jumped, measured in centimeters), and "Rehabilitation_Efficiency_Score" (a score indicating how efficient the rehabilitation was). These variables being important align with our expectations of the data, as inefficient recovery and movement/ balance abnormalities often contribute to injury. What was surprising however was that "Injury_Type" and "Injury_Severity" were not more important variables as some types of injuries are more likely to reoccur and and more severe injury's tend to not heal completely compared to less severe injury, making a player more prone to re-injure themselves.

Principal Component Analysis

```
# PCA recipe for predictors only (unsupervised)
dat_rec = recipe(~., data = injury_dat) |>
  step_naomit(all_predictors()) |>
  step_rm(Player_ID, Injury_Recurrence, Date_of_Injury) |> # removed
  #because not important
  step_normalize(all_numeric_predictors()) |>
  step_dummy(all_nominal_predictors()) |>
  step_pca(all_predictors(), threshold = .95) |> # step 5
  prep()


# Apply PCA
pca_rec = dat_rec |> bake(new_data = NULL)

pca_rec
```

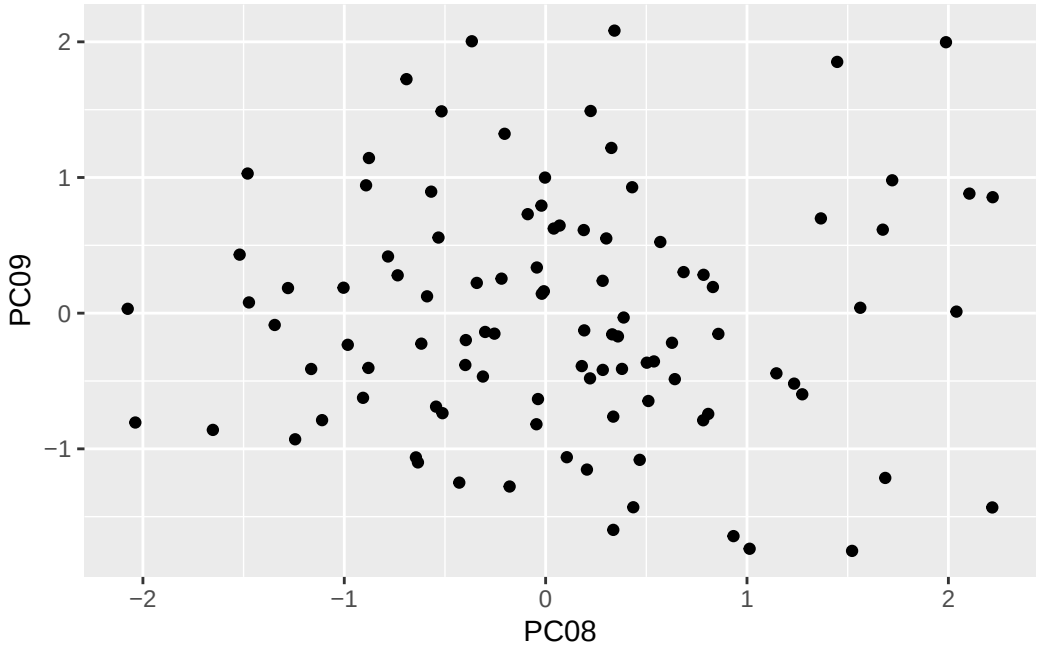
```
# A tibble: 100 x 15
      PC01   PC02    PC03   PC04   PC05     PC06   PC07    PC08   PC09   PC10
     <dbl>  <dbl>   <dbl>  <dbl>  <dbl>    <dbl>  <dbl>   <dbl>  <dbl>  <dbl>
 1 -0.0896  1.89   0.114   0.469  0.753 -0.00202 -0.673 -1.00    0.188 -0.469
 2  0.375  -2.63   0.662   0.855  0.317  0.614   -0.897  0.0404  0.624 -0.679
 3  2.04    1.33  -1.16    1.68   0.378  1.29     0.367 -0.0380 -0.633  0.712
 4 -0.123   0.487 -1.83   -1.45   0.212 -1.82    -1.48   1.37    0.698 -0.128
 5  0.114  -2.58   0.194   0.113  1.42  -0.317   -1.23  -0.634  -1.10   0.422
 6  0.166   0.115 -0.872   0.913 -1.06   1.67    -1.02   0.342   2.08   1.26
 7  2.35    1.13   0.0108  0.461 -1.99   0.893   -1.13  -0.544  -0.689  0.753
 8 -1.26   -0.556  1.58    0.744  0.549  0.492   -0.917  0.502  -0.365  0.254
 9  1.65   -0.231  1.15   -1.25   0.828  0.516   -0.709 -1.16   -0.412 -1.06
10 -0.470   2.20  -1.82   -1.51   1.56  -0.474   -0.169  0.327   1.22   0.112
# i 90 more rows
# i 5 more variables: PC11 <dbl>, PC12 <dbl>, PC13 <dbl>, PC14 <dbl>,
#   PC15 <dbl>
```

```
# Variance explained by components
dat_rec |> tidy(number = 5, type = "variance") |>
  filter(terms == "cumulative percent variance") |>
  print(n = 20)
```

```
# A tibble: 21 x 4
```

```
   terms                       value component id
   <chr>                       <dbl>     <int> <chr>
 1 cumulative percent variance  11.8         1 pca_wDIDr
 2 cumulative percent variance  22.7         2 pca_wDIDr
 3 cumulative percent variance  31.8         3 pca_wDIDr
 4 cumulative percent variance  40.8         4 pca_wDIDr
 5 cumulative percent variance  49.3         5 pca_wDIDr
 6 cumulative percent variance  56.7         6 pca_wDIDr
 7 cumulative percent variance  64.0         7 pca_wDIDr
 8 cumulative percent variance  71.0         8 pca_wDIDr
 9 cumulative percent variance  77.0         9 pca_wDIDr
10 cumulative percent variance  82.2        10 pca_wDIDr
11 cumulative percent variance  86.5        11 pca_wDIDr
12 cumulative percent variance  89.2        12 pca_wDIDr
13 cumulative percent variance  91.4        13 pca_wDIDr
14 cumulative percent variance  93.4        14 pca_wDIDr
15 cumulative percent variance  95.0        15 pca_wDIDr
16 cumulative percent variance  96.5        16 pca_wDIDr
17 cumulative percent variance  97.7        17 pca_wDIDr
18 cumulative percent variance  98.6        18 pca_wDIDr
19 cumulative percent variance  99.3        19 pca_wDIDr
20 cumulative percent variance  99.8        20 pca_wDIDr
# i 1 more row
```

```
pca_rec |> ggplot(aes(x = PC08, y = PC09)) +
  geom_point()
```



Through principal component analysis we could reduce the original predictor set to 15 components while retaining 95.05% of the variance. However the cumulative variance seems to level off at about 11 components which would retain about 86.45% of the variance. Since our data does not have high dimensionality, principal component analysis doesn't need to be applied. If we were looking to reduce dimensionality for a simpler or faster model like logistic regression we could use these PC's as inputs.

Potential Next Steps

Address Overfitting

To try and improve our model we could try to simplify it by reducing the trees and increasing min_n and we could try dimensionalty reduction with PCA before modeling.

Evaluate Other Metrics

Instead of basing our model on accuracy, we could check precision and ROC AUC.